# Optimal Channel Naming for Compositional Rewrite Translations via Set Automaton Partial Evaluation

L.G. Meredith
F1R3FLY.io

May 12, 2026

**Abstract**

We present a translation scheme that compiles graph-structured lambda-theory (GSLT) rewrites into the rho calculus, and we identify the channel name for each rewrite context as the canonical reflection of a state in a Bouwman–Erkens set automaton. Treating the polymorphic compilation step $\mathsf{let}\ tc = [\![ K ]\!]\ \mathsf{in}\ \cdots$ as a partial evaluation of the set automaton against the context $K$ yields a closed-form formula $tc(K) = \ulcorner \delta^*(s_0, \mathsf{surf}(K)) \urcorner$. We then transport the three principal theorems of the Bouwman–Erkens construction — symbol-once inspection, prune-preserves-work, and the coarsest-sound-partition theorem for the outermost-preserving dependency relation — into the channel-based setting. The resulting scheme is optimal in three precise senses: each surface symbol of every redex context is consumed by exactly one for-receive, inner reductions never invalidate outer channels, and the channel quotient induced by $tc(\cdot)$ is the coarsest equivalence on contexts that preserves outermost firing. We discuss the non-linear extension via rho-calculus consistency receives, and outline the changes required to support dynamic rule sets.

## Contents

**1 Introduction** **2**

**2 Preliminaries** **3**
2.1 Terms, contexts, and rewrites . . . 3
2.2 Graph-structured lambda theories . . . 4
2.3 Rho calculus, briefly . . . 5
2.4 Set automata . . . 5

**3 The translation scheme** **8**
3.1 Compilation of terms, rules, and contexts . . . 8
3.2 What "optimal" means here . . . 9

**4 The channel-naming construction** **9**
4.1 From a context to a state . . . 9
4.2 The compiled rule, in detail . . . 10
4.3 Algorithmic construction of $tc$ . . . 10

**5 Optimality** **11**
5.1 Symbol-once inspection . . . . . 11
5.2 Prune-preserves-work . . . . . 11
5.3 Coarsest sound partition . . . . . 11

**6 Worked examples** **12**
6.1 Lambda calculus with head-context reduction . . . . . 12
6.2 The rho calculus into itself . . . . . 14

**7 Non-linear extensions** **16**

**8 Caveats and future work** **17**

**9 Conclusion** **17**

# 1 Introduction

Compiling a rewrite system into a process calculus poses a tension that does not arise when implementing a rewrite engine in an imperative or functional host language. In the latter setting, a rewrite procedure can freely re-traverse the term being rewritten, and an efficient pattern matcher — such as the set automaton of Bouwman and Erkens [1, 2] — ensures that each function symbol is inspected only once per redex search. In the former setting, every traversal step is a communication, every intermediate matching state must be reified as a channel or process, and every redex firing must be expressible as a send-receive pair.

The translation scheme considered here, due to Meredith, compiles a single rewrite rule $L \rightsquigarrow R$ as

$$[\![\, L \rightsquigarrow R \,]\!](\ell\!\ell) \;=\; \mathsf{for}\,([\![\, L \,]\!] \Leftarrow \ell\!\ell)\,\{\; \ell\!\ell\,!\,([\![\, R \,]\!])\;\} \tag{1}$$

and a contextual rewrite as

$$\begin{aligned}[\![\, L_1 \rightsquigarrow R_1, \ldots, L_n \rightsquigarrow R_n \;\Rightarrow\; K(L_1,\ldots,L_n) \rightsquigarrow K'(R_1,\ldots,R_n) \,]\!] \\ =\; \mathsf{let}\; tc = [\![\, K \,]\!]\; \mathsf{in}\; \mathsf{for}\,(([\![\, L_1 \,]\!],\ldots,[\![\, L_n \,]\!]) \Leftarrow tc)\,\{\; tc\,!\,([\![\, K' \,]\!]([\![\, R_1 \,]\!],\ldots,[\![\, R_n \,]\!]))\;\}\end{aligned} \tag{2}$$

The polymorphic interpretation of $\mathsf{let}\; tc = [\![\, K \,]\!]\; \mathsf{in}\; \cdots$ is the locus of optimisation. Since $K$ is a context (a term with $n$ holes), the channel $tc$ is a function of $K$'s shape; the question is which function makes the overall translation optimal.

We argue that the right answer is given by partial evaluation of a set automaton built from the left-hand sides $\{L_1, \ldots, L_n\}$. Concretely:

$$tc(K) \;=\; \ulcorner \delta^*(s_0, \mathsf{surf}(K)) \urcorner,$$

where $(\mathcal{S}, s_0, L, \delta, \eta)$ is the set automaton for the pattern set, $\mathsf{surf}(K)$ is the function-symbol skeleton of $K$ in the inspection order prescribed by the state-labelling function $L$, and $\ulcorner \cdot \urcorner$ is the rho-calculus reflection of an automaton state into a channel name. Two contexts $K$ and $K'$ that lead the automaton to the same state are then assigned the same channel; two contexts that the automaton distinguishes are assigned distinct channels. We show that the partition on contexts induced by $tc(\cdot)$ is the coarsest partition sound for outermost firing.

**Contributions.**

- A formal definition of the channel-naming function $tc(K)$ as a partial evaluation of a set automaton (§4).
- Three optimality theorems transported from [1]: symbol-once inspection (Theorem 5.1), prune-preserves-work (Theorem 5.2), and coarsest sound partition (Theorem 5.3).
- A treatment of the non-linear case (§7) via consistency receives, paralleling the Bouwman–Erkens *ambiguous/disabled/enabled* bookkeeping.
- A discussion of dynamic rule sets and other engineering caveats (§8).

**Outline.** Section 2 fixes notation for the rho calculus, GSLT rewrites, and set automata. Section 3 reviews the translation scheme and isolates the polymorphic step. Section 4 gives the channel-computation algorithm. Section 5 proves the optimality theorems. Section 7 treats non-linearity. Section 8 discusses caveats and future work.

# 2 Preliminaries

## 2.1 Terms, contexts, and rewrites

Fix a finite ranked alphabet $\mathbb{F}$ with arity map $\#\cdot : \mathbb{F} \to \mathbb{N}$ and a countable set of variables $V$, with $\mathbb{F} \cap V = \emptyset$. The set $\mathcal{T}(\mathbb{F}, V)$ of terms is generated by $V \subseteq \mathcal{T}(\mathbb{F}, V)$ together with the formation rule that $f(t_1, \ldots, t_n) \in \mathcal{T}(\mathbb{F}, V)$ whenever $f \in \mathbb{F}$ has $\#f = n$ and $t_1, \ldots, t_n \in \mathcal{T}(\mathbb{F}, V)$. We write $vars(t)$ for the set of variables occurring in $t$ and call $t$ *ground* when $vars(t) = \emptyset$.

A *position* is a finite list of positive integers; we write $p, q \in \mathbb{P}$, $\epsilon$ for the empty list (the *root position*), and $p \cdot q$ for concatenation, satisfying $\epsilon \cdot p = p \cdot \epsilon = p$. The *prefix order* $p \leq q$ holds iff $p \cdot r = q$ for some $r$. The *domain* of a term is defined by $\mathcal{D}(x) = \{\epsilon\}$ for $x \in V$ and $\mathcal{D}(f(t_1, \ldots, t_n)) = \{\epsilon\} \cup \{\, i \cdot p \mid 1 \leq i \leq n,\ p \in \mathcal{D}(t_i) \,\}$. For $p \in \mathcal{D}(t)$, the subterm at $p$ is $t|_p$, with $t|_\epsilon = t$ and $f(t_1, \ldots, t_n)|_{i \cdot p'} = t_i|_{p'}$; the replacement $t[u]_p$ is the unique term obtained from $t$ by replacing $t|_p$ with $u$. The *edge set* $\mathcal{E}(t) \subseteq \mathcal{D}(t)$ is the set of positions of variable occurrences, and $\mathcal{D}_{\setminus\mathcal{E}}(t) = \mathcal{D}(t) \setminus \mathcal{E}(t)$ is the set of function-symbol positions. For a non-variable term $t = f(t_1, \ldots, t_n)$ the *head symbol* is $\mathsf{hd}(t) = f$.

A *pattern* is a non-variable term; a pattern is *linear* if no variable occurs in it more than once. A *substitution* is a mapping $\sigma : V \to \mathcal{T}(\mathbb{F}, V)$; we write $t^\sigma$ for its homomorphic extension applied to $t$.

A *context* of arity $n$ is a term-with-holes $K(\Box_1, \ldots, \Box_n)$ in which each hole $\Box_i$ occurs exactly once. We write $H_K \subseteq \mathcal{D}(K)$ for the set of hole positions and $K[t_1, \ldots, t_n]$ for the term obtained by simultaneous substitution of $t_i$ for $\Box_i$.

A *rewrite rule* $L \rightsquigarrow R$ is a pattern $L$ paired with a term $R$ such that $vars(R) \subseteq vars(L)$. A *contextual rewrite* is a sequent

$$L_1 \rightsquigarrow R_1, \ldots, L_n \rightsquigarrow R_n \ \Rightarrow\ K(L_1, \ldots, L_n) \rightsquigarrow K'(R_1, \ldots, R_n),$$

asserting that, in any context where the inner rewrites are applicable, the outer pattern reduces to the outer right-hand side.

## 2.2 Graph-structured lambda theories

We adopt the semantic definition of a graph-structured lambda theory and treat term-rewriting data as a finite *presentation* of one.

**Definition 2.1** (Graph-structured lambda theory)**.** A *graph-structured lambda theory* (GSLT) is a tuple $\mathcal{G} = (\mathcal{C}, P, R)$ where

- $\mathcal{C}$ is a category with finite limits;
- $\mathcal{C}$ is closed with respect to the cartesian product (i.e. for every object $A$ the functor $(-) \times A$ has a right adjoint $A \Rightarrow (-)$, giving internal homs);
- $P \in \mathcal{C}$ is a distinguished object, called the *object of processes*;
- $R \hookrightarrow P \times P$ is a distinguished subobject, called the *reduction relation*.

A *syntactic morphism* of GSLTs $\mathcal{G} \to \mathcal{G}'$ is a functor $F : \mathcal{C} \to \mathcal{C}'$ that strictly preserves finite limits and internal homs, satisfies $F(P) = P'$ on the nose, and restricts on $R$ to a map into $R'$ over $P' \times P'$.

The reduction relation $R$ is given as an abstract subobject; in practice the GSLTs of interest are obtained as the free GSLT generated by a finite collection of generators and relations. Term rewriting provides one such presentation.

**Definition 2.2** (Finite presentation)**.** A *finite presentation* of a GSLT consists of a finite ranked alphabet $\mathbb{F}$ (and an apparatus for binders, if needed) together with a finite set

$$\mathcal{R} \;=\; \mathcal{R}_{\text{rule}} \cup \mathcal{R}_{\text{ctx}}$$

of rewrite rules and contextual rewrites in the sense above.

**Construction 2.3** (GSLT generated by a presentation)**.** Let $\mathcal{R}$ be a presentation over $\mathbb{F}$. The *generated GSLT* $\mathcal{G}(\mathcal{R}) = (\mathcal{C}_{\mathbb{F}}, P, R(\mathcal{R}))$ is built as follows.

- $\mathcal{C}_{\mathbb{F}}$ is the cartesian closed category freely generated by the signature: objects are simple types built from a base sort $\iota$ under $\times$ and $\Rightarrow$, and morphisms are $\lambda$-terms modulo $\beta\eta$ over the function symbols of $\mathbb{F}$ (equivalently, the syntactic category of the simply-typed $\lambda$-theory whose constants are $\mathbb{F}$). This category has all finite limits and is cartesian closed by construction.
- $P$ is the base sort $\iota$; ground terms of $\mathcal{T}(\mathbb{F})$ are the global elements $1 \to P$.
- $R(\mathcal{R}) \hookrightarrow P \times P$ is the *inductively generated* subobject specified in Definition 2.4 below.

**Definition 2.4** (Inductively generated reduction relation)**.** $R(\mathcal{R}) \hookrightarrow P \times P$ is the smallest subobject closed under the following clauses, read pointwise on global elements (equivalently, on ground terms over $\mathbb{F}$):

**(Rule)**
For every rewrite rule $L \rightsquigarrow R$ in $\mathcal{R}_{\text{rule}}$ and every ground substitution $\sigma$, $(L^\sigma, R^\sigma) \in R(\mathcal{R})$.

**(Context)**
For every contextual rewrite $L_1 \rightsquigarrow R_1, \ldots, L_n \rightsquigarrow R_n \Rightarrow K(L_1, \ldots, L_n) \rightsquigarrow K'(R_1, \ldots, R_n)$ in $\mathcal{R}_{\text{ctx}}$ and every ground substitution $\sigma$: if $(L_i^\sigma, R_i^\sigma) \in R(\mathcal{R})$ for each $1 \le i \le n$, then $\big(K^\sigma(L_1^\sigma, \ldots, L_n^\sigma),\, K'^\sigma(R_1^\sigma, \ldots, R_n^\sigma)\big) \in R(\mathcal{R})$.

Equivalently, $R(\mathcal{R})$ is the least fixed point of the monotone operator on subobjects of $P \times P$ that adds the $\sigma$-instances of the rule clauses and the $\sigma$-instances of the context clauses whose premises already lie in the argument.

*Remark* 2.5. The GSLT proper is the generated $\mathcal{G}(\mathcal{R})$ of Construction 2.3. By a mild abuse we write $\mathcal{R}$ for both the presentation and the GSLT it generates wherever no confusion can arise; statements about "redexes," "firing," and "the reduction relation" should be read as statements about $R(\mathcal{R})$.

The set of *left-hand sides* of a presentation $\mathcal{R}$ is

$$\mathcal{L} \;=\; \{L \mid L \rightsquigarrow R \in \mathcal{R}_{\text{rule}}\} \cup \{L_i \mid L_1 \rightsquigarrow R_1, \ldots, L_n \rightsquigarrow R_n \Rightarrow \cdots \in \mathcal{R}_{\text{ctx}},\ 1 \le i \le n\}.$$

The set automaton of §2.4 is built from $\mathcal{L}$ alone; it is the syntactic data of the presentation, and is the same for any two presentations that share their left-hand sides.

## 2.3 Rho calculus, briefly

We use a minimal fragment of the rho calculus [4]. Processes $P$ and names $x$ are mutually defined by

$$P \;::=\; \mathbf{0} \;\mid\; x!(P) \;\mid\; \mathsf{for}\,(y \Leftarrow x)\,\{P\} \;\mid\; P \mid P \;\mid\; *x, \qquad x \;::=\; @P,$$

where $\mathbf{0}$ is the inert process, $x\,!\,(P)$ is an asynchronous send of $P$ on channel $x$, $\mathsf{for}\,(y \Leftarrow x)\,\{\,P\,\}$ is a persistent receive on channel $x$ that binds the incoming name to $y$ in $P$, $P \mid Q$ is parallel composition, $@P$ is the *reflection* of process $P$ to a name, and $*x$ is the *dereference* of name $x$ to a process.

The rho calculus has a structural congruence $\equiv$ that includes $\alpha$-equivalence, associativity-commutativity of $\mid$ with $\mathbf{0}$ as unit, and the reflection law $*@P \equiv P$. Names are quotients of processes under $\equiv$: two names $x = @P$ and $x' = @P'$ are *name-equal*, written $x =_{\mathcal{N}} x'$, iff $P \equiv P'$. The principal reduction rule (COMM) is

$$\mathsf{for}\,(y \Leftarrow x)\,\{\,P\,\} \;\mid\; x'\,!\,(Q) \;\rightsquigarrow\; P[@Q/y] \qquad \text{whenever } x =_{\mathcal{N}} x',$$

with the persistent variant leaving a fresh copy of the receive in place.

We use $\mathsf{let}\ x = P\ \mathsf{in}\ Q$ as syntactic sugar for $Q[@P/x]$ when $P$ is a name-valued expression. Tuple-pattern receives $\mathsf{for}\,((y_1, \ldots, y_n) \Leftarrow x)\,\{\,P\,\}$ are the standard pattern-match on incoming names, decomposable into a finite number of nested single-name receives guarded by name equality (an explicit elaboration is unnecessary for what follows).

The reflection bracket $\ulcorner\cdot\urcorner$, used in the body of the paper, is a section of $@\cdot$ chosen so that distinguishable mathematical objects are mapped to distinct names: $\ulcorner a \urcorner =_{\mathcal{N}} \ulcorner b \urcorner$ iff $a$ and $b$ are equal as mathematical objects. For automaton states $s \in \mathcal{S}$, $\ulcorner s \urcorner$ denotes a canonical name unique to $s$; we extend this convention to configuration trees in §4.

## 2.4 Set automata

We summarise the construction of [1, 2] in enough detail to make the rest of the paper self-contained. Readers already familiar with their work may skim this subsection.

**The tuple.** A *set automaton* for a finite pattern set $\mathcal{L}$ over $\mathbb{F}$ is a tuple

$$M \;=\; (\mathcal{S},\, s_0,\, L,\, \delta,\, \eta),$$

where $\mathcal{S}$ is a finite set of *states*, $s_0 \in \mathcal{S}$ is the *initial state*, $L : \mathcal{S} \to \mathbb{P}$ is the *state-labelling* function (it selects the next position to inspect from a configuration in state $s$), $\delta : \mathcal{S} \times \mathbb{F} \to \mathcal{P}(\mathcal{S} \times \mathbb{P})$ is the *transition* function, and $\eta : \mathcal{S} \times \mathbb{F} \to \mathcal{P}(\mathcal{L} \times \mathbb{P})$ is the *output* function.

**State encoding: match goals.** States are encoded as non-empty sets of *match goals*. Write $sub(\ell) = \{\, \ell|_p \mid p \in \mathcal{D}_{\setminus\mathcal{E}}(\ell) \,\}$ for the non-variable subpatterns of $\ell$, and $sub(\mathcal{L}) = \bigcup_{\ell \in \mathcal{L}} sub(\ell)$. The set of *match obligations* is $MO = \mathcal{P}(sub(\mathcal{L}) \times \mathbb{P}) \setminus \{\emptyset\}$, and the set of *match announcements* is $MA = \mathcal{L} \times \mathbb{P}$. A *match goal* is a pair $(mo, ma) \in MO \times MA$, written

$$\ell_1 @ p_1, \ldots, \ell_n @ p_n \;\hookrightarrow\; \ell @ p,$$

and read: "to announce a match for $\ell$ at position $p$, it remains to observe each subpattern $\ell_i$ at position $p_i$". A goal of the form $\ell @ p \hookrightarrow \ell @ p$ is *fresh*; one of the form $mo \hookrightarrow \ell @ \epsilon$ is a *root goal*. States are non-empty subsets of $MO \times MA$, and the initial state collects the fresh root goals for every left-hand side:

$$s_0 \;=\; \{\, \ell @ \epsilon \hookrightarrow \ell @ \epsilon \mid \ell \in \mathcal{L} \,\}.$$

The state-labelling $L$ must select, for each state $s$, a position appearing in some root goal of $s$; this forces $L(s_0) = \epsilon$. In general the choice is not unique; we fix any canonical policy (§8).

**Derivative.** Let $\mathsf{pos}(mo) = \{\, q \mid \ell @ q \in mo \,\}$. Given a state $s$, a symbol $f$ with $n = \#f$, and the position $p = L(s)$, the *reduce* operation on a match obligation is

$$\mathsf{reduce}(mo, f, p) \;=\; \{\, \ell @ q \in mo \mid q \neq p \,\} \;\cup\; \{\, \ell|_i @\, p \cdot i \mid \ell @ p \in mo,\; 1 \le i \le n,\; \ell|_i \notin V \,\},$$

applied when $\ell @ p \in mo$ for some $\ell$ with $\mathsf{hd}(\ell) = f$, and yielding $\emptyset$ if no such $\ell$ exists. The *$f$-derivative* of $s$ is

$$\mathsf{deriv}(s, f) \;=\; \mathit{reduced} \cup \mathit{unchanged} \cup \mathit{fresh},$$

where

$$\begin{aligned}
\mathit{reduced} \;&=\; \{\, \mathsf{reduce}(mo, f, L(s)) \hookrightarrow ma \mid mo \hookrightarrow ma \in s, \\
&\qquad\qquad \exists \ell @ L(s) \in mo.\, \mathsf{hd}(\ell) = f,\; \mathsf{reduce}(mo, f, L(s)) \neq \emptyset \,\}, \\
\mathit{unchanged} \;&=\; \{\, mo \hookrightarrow ma \in s \mid L(s) \notin \mathsf{pos}(mo) \,\}, \\
\mathit{fresh} \;&=\; \{\, \ell|_i @\, L(s) \cdot i \hookrightarrow \ell|_i @\, L(s) \cdot i \mid \ell \in \mathcal{L},\; 1 \le i \le n,\; \ell|_i \notin V \,\}.
\end{aligned}$$

*Reduced* goals are those whose obligation refers to $L(s)$ and whose head there matches $f$; *unchanged* goals do not refer to $L(s)$; *fresh* goals re-seed redex search at the children of $L(s)$. The output function records completed reductions:

$$\eta(s, f) \;=\; \{\, ma \in MA \mid f(x_1, \ldots, x_n) @\, L(s) \hookrightarrow ma \in s,\; x_1, \ldots, x_n \in V \,\}.$$

**Partitioning by dependency; lifting by greatest common prefix.** The naive choice $\delta(s, f) = \{(\mathsf{deriv}(s, f), \epsilon)\}$ has infinitely many states, as positions grow without bound. Two finitising steps cure this: partition $\mathsf{deriv}(s, f)$ into mutually-independent classes, and shift each class back to a canonical origin.

The *direct dependency* relation $R_{\mathsf{dep}}$ on match goals is

$$\big(mo_1 \hookrightarrow ma_1\big)\ R_{\mathsf{dep}}\ \big(mo_2 \hookrightarrow ma_2\big) \quad \Longleftrightarrow \quad \mathsf{pos}(mo_1) \cap \mathsf{pos}(mo_2) \neq \emptyset.$$

$R_{\mathsf{dep}}$ is reflexive and symmetric but not in general transitive; let $\sim$ be its transitive closure. Partition $\mathsf{deriv}(s, f)$ into $\sim$-equivalence classes; write $[\mathsf{deriv}(s, f)]_\sim$ for the set of classes. For a class $K$, all match-announcement and obligation positions in $K$ share a (possibly empty) greatest common prefix $\mathsf{gcp}(K)$. The *lift* of $K$ rewrites each position $\mathsf{gcp}(K) \cdot p'$ appearing in $K$ to $p'$:

$$\mathsf{lift}(K) \ = \ \{\, \{\ell_i @\, p_i'\}_i \hookrightarrow \ell @\, q' \mid \{\ell_i @\, \mathsf{gcp}(K) \cdot p_i'\}_i \hookrightarrow \ell @\, \mathsf{gcp}(K) \cdot q' \in K \,\}.$$

The transition function is then

$$\delta(s, f) \ = \ \{\, (\mathsf{lift}(K),\, \mathsf{gcp}(K)) \mid K \in [\mathsf{deriv}(s, f)]_\sim \,\},$$

read: "from configuration $(s, p)$ observing $f$, advance to $(\mathsf{lift}(K), p \cdot \mathsf{gcp}(K))$ for each class $K$". Only finitely many lifted classes can ever arise, so the construction terminates. We write $\delta^*(s, w)$ for the iterated transition along a sequence $w \in \mathbb{F}^*$ of observations.

**Outermost-preserving partition.** The partition by $\sim$ yields the smallest set automaton sound for arbitrary strategies. For an outermost strategy it is too fine: it separates goals one would want kept together to discover an outermost match before its deeper inner matches [1, §7]. The *outermost-preserving* relation $R_{\mathsf{op}}$ enlarges $R_{\mathsf{dep}}$ by also relating goals whose match-announcement positions are prefix-comparable:

$$g_1\ R_{\mathsf{op}}\ g_2 \quad \Longleftrightarrow \quad g_1\ R_{\mathsf{dep}}\ g_2 \text{ or } p_1 \leq p_2 \text{ or } p_2 \leq p_1,$$

where $g_i = mo_i \hookrightarrow \ell_i @\, p_i$. Replacing $\sim$ by the transitive closure of $R_{\mathsf{op}}$ throughout the construction yields a (typically larger but still finite) automaton in which a depth-first traversal of the configuration tree yields outermost matches first. We write $M_{R_{\mathsf{dep}}}$ and $M_{R_{\mathsf{op}}}$ for the two automata when the distinction matters.

**Configurations and configuration trees.** Running $M$ on a ground term $t$ traverses positions of $t$ under the guidance of $L$ and $\delta$. A *configuration* is a pair $(s, p) \in \mathcal{S} \times \mathbb{P}$; from it the procedure inspects the symbol $\mathsf{hd}(t|_{p \cdot L(s)})$ to follow $\delta$. The progress of a traversal is recorded in a *configuration tree*

$$CT \ ::= \ \mathsf{B}(s, p) \ \mid \ \mathsf{N}(s, p, cts),$$

where a *bud* $\mathsf{B}(s, p)$ is an unexplored configuration and a *node* $\mathsf{N}(s, p, cts)$ is an explored configuration with a finite (possibly empty) set $cts$ of child configuration trees. The set of nodes of $ct$ is

$$\mathsf{nodes}(\mathsf{B}(s, p)) = \emptyset, \qquad \mathsf{nodes}(\mathsf{N}(s, p, cts)) = \{(s, p)\} \cup \bigcup_{ct' \in cts} \mathsf{nodes}(ct').$$

The *completed configuration tree* of a ground term $t$ is

$$\mathsf{completed}(s, p, t) \ = \ \mathsf{N}\big(s,\ p,\ \{\, \mathsf{completed}(s', p \cdot p', t) \mid (s', p') \in \delta(s, \mathsf{hd}(t|_{p \cdot L(s)})) \,\}\big),$$

with $\mathsf{completed}(t) = \mathsf{completed}(s_0, \epsilon, t)$. The *grow* and *prune* operations are

$$\begin{aligned}
\mathsf{grow}(\mathsf{B}(s,p), t) &= \mathsf{N}\big(s,\ p,\ \{\, \mathsf{B}(s', p \cdot p') \mid (s', p') \in \delta(s, \mathsf{hd}(t|_{p \cdot L(s)})) \,\}\big), \\
\mathsf{prune}(\mathsf{N}(s,p,cts), q) &= \begin{cases} \mathsf{B}(s,p) & \text{if } p \cdot L(s) = q, \\ \mathsf{N}(s,p,\{\, \mathsf{prune}(ct', q) \mid ct' \in cts \,\}) & \text{otherwise,} \end{cases} \\
\mathsf{prune}(\mathsf{B}(s,p), q) &= \mathsf{B}(s,p).
\end{aligned}$$

For a redex at position $q$ inside $t$, there is a unique node of $\mathsf{completed}(t)$ — the *initialisation configuration* of the redex — whose position is $q$; $\mathsf{prune}(\cdot, q)$ collapses that node back to a bud.

**Three results we will lean on.** The following statements are proved in [1, Thm. 1, Thm. 2, Lemma 1]. We restate them so that §5 can cite them locally.

**Theorem 2.6** (Symbol-once bijection [1, Thm. 1])**.** *For any set automaton $M$ for $\mathcal{L}$ and any ground term $t$, the map*

$$\mathsf{N}(s,p,cts) \longmapsto p \cdot L(s)$$

*is a bijection between the nodes of* $\mathsf{completed}(t)$ *and the domain $\mathcal{D}(t)$. In particular,* $|\mathsf{nodes}(\mathsf{completed}(t))| = |\mathcal{D}(t)|$.

**Theorem 2.7** (Matching correctness [1, Thm. 2])**.** *Define* $\mathsf{matches}(s,p,t) = \{\, (\ell @ p \cdot q) \mid (\ell, q) \in \eta(s, \mathsf{hd}(t|_{p \cdot L(s)})) \,\}$. *The set of all redexes of a ground term $t$ for $\mathcal{L}$ is*

$$\bigcup \{\, \mathsf{matches}(s,p,t) \mid (s,p) \in \mathsf{nodes}(\mathsf{completed}(t)) \,\}.$$

**Lemma 2.8** (Prune properties [1, Lemma 1])**.** *Let $\sqsubseteq$ be the* fragment-of *order on configuration trees, generated by* $\mathsf{B}(s,p) \sqsubseteq \mathsf{N}(s,p,cts)$ *and closed componentwise on children. Then for any $ct \sqsubseteq ct'$ and any position $q$:*

(i) $\mathsf{prune}(ct, q) \sqsubseteq ct$ *(pruning never adds nodes);*

(ii) $\mathsf{prune}(ct, q) \sqsubseteq \mathsf{prune}(ct', q)$ *(pruning is monotone in the fragment order);*

(iii) *if* $t \xrightarrow{(\ell \rightsquigarrow r)@q} t'$, *then* $\mathsf{prune}(\mathsf{completed}(t), q) = \mathsf{prune}(\mathsf{completed}(t'), q)$ *(firing preserves the trimmed tree); and*

(iv) *the nodes of $ct$ strictly outside the subtree rooted at $q$ are preserved by* $\mathsf{prune}(\cdot, q)$.

# 3 The translation scheme

## 3.1 Compilation of terms, rules, and contexts

We write $[\![ \cdot ]\!]$ for the compilation function from terms to processes (or to names, depending on context); we leave its base cases on function symbols and variables abstract, since the optimality argument is parametric in them. The operative clauses are (1)–(2) from the introduction, which we restate in a form convenient for the analysis.

**Definition 3.1** (Compilation of rewrite rules)**.** For a single rule $L \rightsquigarrow R$ on translation channel $\mathit{tt}$,

$$[\![\, L \rightsquigarrow R \,]\!](\mathit{tt}) \;=\; \mathsf{for}\,([\![\, L \,]\!] \Leftarrow \mathit{tt})\,\{\ \mathit{tt}\,!\,([\![\, R \,]\!])\ \}.$$

For a contextual rewrite,

$$[\![\, L_1 \rightsquigarrow R_1, \ldots, L_n \rightsquigarrow R_n \;\Rightarrow\; K \rightsquigarrow K' \,]\!] \\ = \; \mathsf{let}\; tc = [\![\, K \,]\!]\; \mathsf{in}\; \mathsf{for}\,((([\![\, L_1 \,]\!], \ldots, [\![\, L_n \,]\!]) \Leftarrow tc)\,\{\; tc\,!\,([\![\, K' \,]\!]([\![\, R_1 \,]\!], \ldots, [\![\, R_n \,]\!]))\;\}\,.$$

The *polymorphic interpretation* of $\mathsf{let}\; tc = [\![\, K \,]\!]\; \mathsf{in}\; \cdots$ is this: $[\![\, K \,]\!]$ is a function only of the surface shape of $K$, not of the hole-fillers. Holes are deliberately absent from the channel formula; they are precisely the data communicated on $tc$ when the rule fires.

### 3.2 What "optimal" means here

Three properties characterise an optimal translation:

**(O1) Symbol-once.** Each function symbol of $K$, whenever the rule fires, is processed by exactly one for-receive in the translation.

**(O2) Prune-preserves.** When an inner contextual rewrite fires *below* the position $p$ of the surrounding $K$, the channels and pending receives strictly outside $p$ remain valid; no re-establishment of the outer channel structure is needed.

**(O3) Coarsest sound.** If two contexts $K$, $K'$ are observationally equivalent — in the sense that no choice of hole-filling distinguishes which rule of $\mathcal{R}$ fires — they share a channel; if they are observationally distinct, they receive distinct channels.

A naive choice such as $tc(K) = @K$ (reflect the syntactic context verbatim) trivially satisfies (O3) but fails (O1): two syntactically distinct contexts that match the same patterns receive different channels and so duplicate the for-receive work. Choices that collapse too many contexts, e.g. $tc(K) = @\mathsf{hd}(K)$, satisfy (O1) cheaply but fail (O3). The set automaton sits exactly at the equivalence quotient that satisfies all three.

## 4 The channel-naming construction

### 4.1 From a context to a state

**Definition 4.1** (Surface of a context)**.** Let $K$ be an $n$-ary context with hole positions $H_K \subseteq \mathcal{D}(K)$. The *surface* of $K$ is the partial term obtained from $K$ by collapsing each hole to a fresh wildcard: $\mathsf{surf}(K)$ has the same function-symbol positions as $K$, with $\mathsf{hd}(\mathsf{surf}(K)|_p) = \mathsf{hd}(K|_p)$ for $p \in \mathcal{D}_{\setminus\mathcal{E}}(K) \setminus H_K$, and is undefined on $H_K$.

**Definition 4.2** (Automaton trace of a context)**.** Let $M = (\mathcal{S}, s_0, L, \delta, \eta)$ be a set automaton for the left-hand-side set of $\mathcal{R}$. The *automaton trace* of a context $K$ is the unique configuration tree $T_M(K)$ obtained by running the matching procedure of [1, §3] on $\mathsf{surf}(K)$, suspended at every configuration $(s, p)$ whose inspection position $p \cdot L(s)$ lies in a hole $H_K$.

The trace $T_M(K)$ is a finite tree because $\mathsf{surf}(K)$ is finite and the automaton is deterministic at every transition step.

**Definition 4.3** (Channel of a context)**.** Define

$$tc(K) \;=\; \ulcorner\, T_M(K) \,\urcorner,$$

the canonical reflection of the suspended automaton trace.

In the unhybridised case where $K$ has a single hole and the automaton is deterministic enough that $T_M(K)$ collapses to a single suspended configuration $(s, p)$, the formula simplifies to $tc(K) = \ulcorner \delta^*(s_0, \mathsf{surf}(K)) \urcorner$, which is the slogan quoted in the introduction. The general definition in terms of the suspended configuration tree is the technically correct one, accommodating $R_{\mathsf{dep}}$-induced parallelism among independent holes and the look-ahead phenomenon noted in [1, §1.1].

### 4.2 The compiled rule, in detail

Substituting Definition 4.3 into Definition 3.1, the contextual rule compiles to

$$\mathsf{let}\ tc = \ulcorner T_M(K) \urcorner\ \mathsf{in}\ \ \mathsf{for}\,((\llbracket L_1 \rrbracket, \ldots, \llbracket L_n \rrbracket) \Leftarrow tc) \qquad \{\, tc\,!\,(\llbracket K' \rrbracket(\llbracket R_1 \rrbracket, \ldots, \llbracket R_n \rrbracket))\,\}\,.$$

The for-receive on $tc$ blocks until $n$ names arrive, one per hole. Their arrival corresponds, automaton-side, to taking the suspended transitions and continuing matching on the hole-fillers. The send on $tc$ in the body emits the rewritten right-hand side, ready to be received by another contextual rule whose hole sits where this one's outer pattern fires.

**Example 4.4** (Boolean simplification)**.** Take $\mathcal{R}$ to be the running example of [1, Ex. 8]:

$$R_1 : \mathit{if}(\mathsf{true}, x, y) \rightsquigarrow x \qquad R_2 : \mathit{if}(\mathsf{false}, x, y) \rightsquigarrow y$$
$$R_3 : \mathit{not}(\mathsf{true}) \rightsquigarrow \mathsf{false} \qquad R_4 : \mathit{not}(\mathsf{false}) \rightsquigarrow \mathsf{true}$$
$$R_5 : \mathit{not}(\mathit{not}(x)) \rightsquigarrow x$$

The set automaton has four states $s_0, s_1, s_2, s_3$ with $L(s_0) = \epsilon$, $L(s_1) = 1$, $L(s_2) = 1$, and the transitions displayed in [1, Fig. 3].

Consider the context $K = \mathit{if}(\Box, \mathsf{false}, \mathsf{true})$ in a contextual rewrite whose inner rule is $R_5$ (so the hole is at position 1). Running the automaton on $\mathsf{surf}(K)$:

$$(s_0, \epsilon) \xrightarrow{\mathit{if}} (s_1, \epsilon) \xrightarrow{\Box} \text{suspend}.$$

We have $tc(K) = \ulcorner (s_1, \epsilon) \urcorner$, a single-configuration trace. The compiled rule listens on this channel for an *if*-headed sub-context filler, performs whatever inner work is needed to bind $x$, and sends the result back on $tc$.

Now consider the context $K' = \mathit{if}(\Box, e, e)$, where the two $\Box$-positioned arguments are required to coincide (a non-linear hole pattern). The set-automaton trace is the same up to position 1, but the hole-equality constraint is communicated on a side-channel; see §7.

### 4.3 Algorithmic construction of $tc$

We summarise the calculation as a procedure (Construction 4.5) for clarity. Pre-conditions: a fixed set automaton $M$ for $\mathcal{R}$ has been constructed off-line.

**Construction 4.5** (Computing $tc(K)$)**.** 1. Initialise the configuration tree $T \leftarrow \mathsf{B}(s_0, \epsilon)$.

2. While $T$ contains a bud $\mathsf{B}(s, p)$ with $p \cdot L(s) \notin H_K$:

   2.1 Let $f = \mathsf{hd}(\mathsf{surf}(K)|_{p \cdot L(s)})$.

   2.2 Replace the bud by $\mathsf{N}(s, p, \{\mathsf{B}(s', p \cdot p') \mid (s', p') \in \delta(s, f)\})$.

3. Return $\ulcorner T \urcorner$.

The loop terminates because each iteration either grows a bud into a node with strictly more nodes-of-context-positions, or transitions into the empty successor set; the number of iterations is bounded by $|\mathcal{D}_{\setminus\mathcal{E}}(\mathsf{surf}(K))|$, in agreement with Theorem 2.6. The reflection in step 3 is a syntactic operation on a finite tree of automaton states, hence computable.

# 5 Optimality

We now state and prove the three optimality theorems. All three are transports of theorems from [1] into the channel setting.

## 5.1 Symbol-once inspection

**Theorem 5.1** (Symbol-once)**.** *Let $\rho$ be a rho-calculus term obtained by translating a GSLT $\mathcal{R}$ via Definition 3.1 with $tc(\cdot)$ as in Definition 4.3. For any reduction sequence of $\rho$ that fires a contextual rewrite at outer-context $K$, each function symbol of $K$ is consumed by exactly one* for*-receive among the compiled rules.*

*Proof sketch.* By Construction 4.5, $tc(K)$ is determined by the suspended configuration tree $T_M(K)$, whose nodes are in bijection with $\mathcal{D}_{\setminus\mathcal{E}}(\mathsf{surf}(K))$ by Theorem 2.6. Each node corresponds to one application of step 2.2, which, in the operational reading of the channel scheme, corresponds to one transition through a sub-channel of $tc$ — i.e. one for-receive in the compiled rule. Conversely, no surface symbol of $K$ is observed twice, by the deterministic-transition guarantee of the set-automaton construction. Hence the map (surface symbol of $K$) $\mapsto$ (for-receive that consumes it) is total and injective, and its image equals the set of for-receives in the compiled rule. □

## 5.2 Prune-preserves-work

In the imperative implementation of [1], when a redex fires at position $p$ the configuration tree is *pruned* to the initialisation configuration of the redex; matching done strictly above $p$ is preserved. The rho-calculus analogue is that the channel structure strictly outside the inner-context position remains live across the inner firing.

**Theorem 5.2** (Prune-preserves-work)**.** *Let $\rho$ contain a compiled outer rule $[\![\,\Phi_{\text{outer}}\,]\!]$ with context $K$ and channel $tc(K)$, and a compiled inner rule $[\![\,\Phi_{\text{inner}}\,]\!]$ with context $K_{\text{in}}$ situated at hole position $p$ of $K$. After firing $\Phi_{\text{inner}}$, the process state restricted to channel $tc(K)$ and to all sub-channels of $tc(K)$ that correspond to surface symbols of $K$ at positions $q$ with $q \not\geq p$ is unchanged.*

*Proof sketch.* Translate the prune-correctness lemma (Lemma 2.8) across Construction 4.5. The relevant invariant is clause (i), $\mathsf{prune}(ct, p) \sqsubseteq ct$, i.e. pruning never adds nodes; combined with clause (iv), the nodes outside the subtree rooted at $p$ survive unchanged. In the channel scheme, "adding nodes" would correspond to creating new for-receives outside the prune position. Since no such creation occurs, the for-receives outside $p$ are exactly those that existed before $\Phi_{\text{inner}}$ fired. The send in the inner rule's body discharges its message on $tc(K_{\text{in}})$, which is a sub-channel of $tc(K)$; the outer for-receive then proceeds to match, finding the same surface symbols of $K$ outside $p$ as before. No re-establishment of the outer channel structure is required. □

### 5.3 Coarsest sound partition

The third theorem is the central one. It says that no compilation scheme that compiles different contexts to different channels can collapse more context pairs than $tc(\cdot)$ does, on pain of unsound firing.

**Theorem 5.3** (Coarsest sound)**.** *Let $\sim_{\mathrm{op}}$ be the equivalence on contexts induced by the $R_{\mathsf{op}}$-set-automaton: $K \sim_{\mathrm{op}} K'$ iff $T_{M_{R_{\mathsf{op}}}}(K) = T_{M_{R_{\mathsf{op}}}}(K')$. Then $\sim_{\mathrm{op}}$ is the coarsest equivalence on contexts such that, for every GSLT $\mathcal{R}$, $K \sim_{\mathrm{op}} K'$ implies that for any choice of hole-fillers $t_1, \ldots, t_n$, the set of rules of $\mathcal{R}$ that fire at the outermost position of $K[t_1, \ldots, t_n]$ equals the set of rules that fire at the outermost position of $K'[t_1, \ldots, t_n]$.*

*Proof sketch.* Soundness: if $K \sim_{\mathrm{op}} K'$, the suspended traces coincide, so the suspended automaton states coincide, so by the matching-correctness theorem (Theorem 2.7) the set of redexes announced by the automaton on $K[t_i]$ equals the set on $K'[t_i]$, for every choice of $t_i$. The outermost subset coincides because $R_{\mathsf{op}}$ preserves prefix-comparability of match-announcement positions (§2.4).

Coarsest: suppose $\sim'$ is strictly coarser, i.e. there exist $K \sim' K'$ with $T_{M_{R_{\mathsf{op}}}}(K) \neq T_{M_{R_{\mathsf{op}}}}(K')$. Then the two traces differ at some configuration, so there is some symbol observation sequence that induces different state-transitions. By the construction of $R_{\mathsf{op}}$ (§2.4), this difference corresponds to at least one match goal that is reduced in one trace and not in the other. Choose hole-fillers $t_i$ that complete the partial match in one and not the other (such fillers exist, since the goal-reduction step expects a specific head symbol). The resulting outermost-match sets at $K[t_i]$ and $K'[t_i]$ disagree, contradicting soundness of $\sim'$. □

The use of $R_{\mathsf{op}}$ rather than $R_{\mathsf{dep}}$ in Theorem 5.3 is deliberate: $R_{\mathsf{dep}}$ gives a strictly coarser partition that is sound for *some* strategy but unsound for outermost firing (the very point made in [1, §7], motivating the introduction of $R_{\mathsf{op}}$). For non-outermost strategies, the analogous coarsest-sound theorem holds with $\sim_{\mathrm{op}}$ replaced by the equivalence induced by the strategy-appropriate dependency relation.

## 6 Worked examples

We now present two extended examples that exhibit the construction in characteristic settings. The first is the lambda calculus with weak head reduction, the canonical example of a contextual rewrite. The second is the rho calculus itself compiled into rho — a reflective example that illustrates what the optimality theorems deliver when the object language and the target language coincide.

### 6.1 Lambda calculus with head-context reduction

**The GSLT.** Take the function symbols *app* (arity 2) and *lam* (arity 1, body-binding). Variables of the object language are represented by a single function symbol *var* (arity 1, name position) which we suppress where it would only clutter notation. The reduction theory has one principal rule and one contextual rule:

$$\begin{aligned} (\beta) \quad & app(lam(M), N) \rightsquigarrow M[N/x], \\ (\textsc{Head}) \quad & M \rightsquigarrow M' \;\Rightarrow\; app(M, N) \rightsquigarrow app(M', N). \end{aligned}$$

Here (Head) encodes weak head reduction: we may reduce in the function position of an application, but not in the argument position and not under a *lam*.

The pattern set for the set automaton is $\mathcal{L} = \{\, app(lam(M), N)\,\}$ — a single LHS of depth two. We treat the bound-variable apparatus of $M$ and the substitution $M[N/x]$ as orthogonal to the channel calculation: they are realised by whatever metaprogramming primitive the host implementation provides, and do not figure in the automaton.

**The set automaton.** Tracing the construction of §2.4:

- $s_0$: initial state, $L(s_0) = \epsilon$, with the single root goal $app(lam(M), N)@\epsilon \hookrightarrow app(lam(M), N)@\epsilon$.
- Reading *app* at $\epsilon$ reduces this goal to $lam(M)@1 \hookrightarrow \ell@\epsilon$ (where $\ell = app(lam(M), N)$); the variable $N$ at position 2 is discarded as a residual. Adding fresh root goals at the children of $\epsilon$ and partitioning by $R_{\mathsf{dep}}$ yields two equivalence classes: one containing the position-1 work, one consisting of a fresh root goal at position 2.
- After lifting, we obtain $\delta(s_0, app) = \{(s_{app}, \epsilon), (s_0, 2)\}$. The transition $(s_0, 2)$ continues redex search in the argument position; the transition $(s_{app}, \epsilon)$ continues the root-level $\beta$-attempt.
- $L(s_{app}) = 1$, the unique obligation position of $s_{app}$'s root goal.
- Reading *lam* at position 1 from $s_{app}$ completes the goal — *lam*'s body $M$ is a variable, so the reduce operation produces $\emptyset$ — and announces the match in $\eta(s_{app}, lam)$.

We do not need the rest of the state graph for the channel calculation. The crucial fact is the position label $L(s_{app}) = 1$.

**Computing the channel.** The contextual rewrite (HEAD) has outer context $K = app(\square, N)$, with hole at position 1 and the variable $N$ at position 2. Since $N$ is a schema variable that ranges over arbitrary terms, position 2 is also effectively a hole from the matcher's point of view: no LHS depends on its content.[1] Running Construction 4.5 on $K$:

1. Start with $T \leftarrow \mathsf{B}(s_0, \epsilon)$.
2. Bud $\mathsf{B}(s_0, \epsilon)$: the inspection position is $\epsilon \cdot L(s_0) = \epsilon$, not a hole. Read *app*. Replace by $\mathsf{N}(s_0, \epsilon, \{\mathsf{B}(s_{app}, \epsilon), \mathsf{B}(s_0, 2)\})$.
3. Bud $\mathsf{B}(s_{app}, \epsilon)$: the inspection position is $\epsilon \cdot L(s_{app}) = 1$, a hole. Suspend.
4. Bud $\mathsf{B}(s_0, 2)$: the inspection position is $2 \cdot L(s_0) = 2$, a hole. Suspend.

The suspended trace contains a single non-trivial bud at $(s_{app}, \epsilon)$ awaiting position 1, plus a "redex-search" bud at $(s_0, 2)$ awaiting position 2. The canonical reflection $tc(K) = \ulcorner T_M(K) \urcorner$ encodes both. The compiled rule, eliding the boilerplate, is:

$$\mathsf{let}\ tc = \ulcorner T_M(K) \urcorner\ \mathsf{in}\ \ \mathsf{for}\,(\llbracket\, app(lam(M), N)\,\rrbracket \Leftarrow tc)\,\{\ tc\,!\,(\llbracket\, app(M[N/x], N)\,\rrbracket)\ \}\,.$$

[1] This is the precise meaning of the parenthetical in the theorem of §3.2: the surface of $K$ is what the automaton *actually* consults, not the syntactic rendering of $K$. Schema variables that no LHS inspects are surface holes.

**What optimality delivers.** The interesting consequences arrive when we consider a chain of head $\beta$-redexes such as

$$t_0 \;=\; app(app(app(lam(M_3), N_3), N_2), N_1),$$

where weak head reduction proceeds by three successive $\beta$-firings at progressively deeper positions in the head spine.

- *Symbol-once* (Theorem 5.1). The three *app* symbols of $t_0$ are processed by exactly three for-receives in the channel encoding, regardless of how many $\beta$-firings occur. Naive translation, in which each head-context instance allocates fresh receivers for the application symbols already on the spine, would re-process the outer *app*s after every inner reduction — giving quadratic work in the spine length. The set-automaton encoding keeps it linear.
- *Prune-preserves-work* (Theorem 5.2). After the innermost $\beta$ fires — replacing $app(lam(M_3), N_3)$ with $M_3[N_3/x]$ — the for-receives associated with the outer two *app*s remain live on their channels. They have already consumed their respective *app* symbols and are now blocked waiting for hole content; no re-establishment is needed.
- *Coarsest-sound* (Theorem 5.3). The argument $N$ in $K = app(\Box, N)$ does not appear in any LHS of $\mathcal{L}$, so the suspended trace $T_M(K)$ is the same for every choice of $N$. All instances of (Head) share a single channel structure regardless of argument shape; this is the maximal sharing consistent with the firing semantics.

In short, weak head normalisation of a head-redex chain of length $n$ takes $O(n)$ work in the channel encoding, with sharing across instances of the head-context rule that no syntax-directed translation could achieve.

### 6.2 The rho calculus into itself

**The GSLT.** We now take the source GSLT to be the rho calculus presented as a term rewriting theory, and compile it into the rho calculus itself. The function symbols are the syntactic constructors of rho: *par* (arity 2), *in* (arity 3), *out* (arity 2), *drop* (arity 1), *nil* (arity 0). We write $in(x, y, P)$ for $\mathsf{for}\,(y \Leftarrow x)\,\{\,P\,\}$ and $out(x, P)$ for $x\,!\,(P)$. The reduction theory has the principal Comm rule and two structural contextual rules:

$$\begin{aligned}
&(\textsc{Comm}) && par(in(x, y, P), out(x, Q)) \;\rightsquigarrow\; P[@Q/y],\\
&(\textsc{Par}_L) && M \rightsquigarrow M' \;\Rightarrow\; par(M, Q) \rightsquigarrow par(M', Q),\\
&(\textsc{Par}_R) && M \rightsquigarrow M' \;\Rightarrow\; par(P, M) \rightsquigarrow par(P, M').
\end{aligned}$$

The Comm rule has a non-linear LHS: the channel name $x$ appears in both the *in*- and the *out*-prefix. We treat the non-linearity per §7, with a consistency receive that checks $x =_{\mathcal{N}} x'$.

**The set automaton.** The pattern set is $\mathcal{L} = \{\, par(in(x, y, P), out(x', Q)) \,\}$ (linearised by renaming the second occurrence of $x$, with the equality recorded for non-linear post-checking). We trace the relevant part of the construction:

- $s_0$: initial, $L(s_0) = \epsilon$, with the fresh root goal at $\epsilon$.
- Reading *par* at $\epsilon$ produces the reduced obligations $in(x, y, P)@1 \hookrightarrow \ell@\epsilon$ and $out(x', Q)@2 \hookrightarrow \ell@\epsilon$, plus fresh root goals at positions 1 and 2.

- Crucially, partitioning by $R_{\mathsf{dep}}$ yields *two* equivalence classes: positions 1 and 2 are independent. Lifting gives a transition with two target states — $(s_{in}, 1)$ for the in-prefix work and $(s_{out}, 2)$ for the out-prefix work — which the matcher may explore in either order or in parallel.
- Each of $s_{in}$ and $s_{out}$ has its own local labelling and transitions; details are unilluminating for the channel calculation. The match for the COMM LHS is announced once both branches reach their respective completion states.

The non-linearity check $x =_{\mathcal{N}} x'$ is dispatched, per Definition 7.1, on a side channel.

**Computing the channel.** For the contextual rule ($\text{PAR}_L$), the outer context is $K_L = \mathit{par}(\Box, Q)$ with hole at position 1 and schema variable $Q$ at position 2. As in the lambda example, $Q$ is a schema variable that no LHS inspects *at the outer level* — the only LHS is the COMM pattern, and that pattern's match work at position 2 is initiated only after the entire LHS commits to matching at the root. From the standpoint of ($\text{PAR}_L$)'s own surface, position 2 is a hole. Construction 4.5 yields:

1. Read *par* at $\epsilon$, producing buds at $(s_{in}, 1)$ and $(s_{out}, 2)$.
2. Bud $(s_{in}, 1)$: inspection position is $1 \cdot L(s_{in})$; if the relevant offset is in the hole-region of $K_L$ (which it is, the hole being at position 1), suspend.
3. Bud $(s_{out}, 2)$: inspection position is $2 \cdot L(s_{out})$; this is in the schema-variable region of $K_L$ at the outer level, suspend.

Both branches are suspended, the two suspensions are independent (they arose from distinct $R_{\mathsf{dep}}$-classes), and $tc(K_L) = \ulcorner T_M(K_L) \urcorner$ records both suspended configurations. The symmetric calculation gives $tc(K_R)$ for ($\text{PAR}_R$), with the in/out roles of positions 1 and 2 exchanged.

**What optimality delivers.** This example highlights five distinct benefits, three from the optimality theorems of §5 and two that are specific to the reflective setting.

- *Symbol-once.* In a process pool $P_1 \mid P_2 \mid \cdots \mid P_n$, parsed as a left-associated *par* tree of depth $n-1$, each *par* is consumed by exactly one for-receive. The number of *par*-symbol observations in the meta-evaluator's run is $O(n)$, not the $O(n^2)$ of a naive "try every pair of children" matcher.
- *Concurrent firing.* The $R_{\mathsf{dep}}$-independence of positions 1 and 2 in the COMM LHS means $tc(K_L)$ and $tc(K_R)$ refer to genuinely independent sub-channels. The compiled rules for ($\text{PAR}_L$) and ($\text{PAR}_R$) may fire concurrently — as the rho calculus's own structural-congruence rules already permit — and the channel encoding inherits this concurrency directly from the automaton's partition structure.[2]
- *Prune-preserves-work.* In a deeply nested process tree with multiple simultaneously-enabled COMM redexes, the firing of any one redex does not invalidate the for-receives associated with the surrounding *par* structure. The meta-evaluator does not have to "re-parse" the process tree after each communication step.
- *Coarsest-sound.* All contexts $\mathit{par}(\Box, Q)$ share the channel $tc(K_L)$ regardless of $Q$, and dually for ($\text{PAR}_R$). The redex-discovery work is shared across every site where the corresponding contextual rule could fire.

[2] Recall the forward-looking remark of §8 on parallel firing: this example is the cleanest setting in which to observe that $R_{\mathsf{dep}}$ is doing the work of a concurrency analysis.

- *Reflective coherence.* The meta-evaluator's channel structure mirrors the object-level rho calculus's channel structure: outer *par*s correspond to outer channels, inner COMM redexes correspond to inner channels, and substitution is realised by sending and receiving names. The optimality of the matcher is therefore inherited as the optimality of the meta-evaluator's parallel structure — precisely the coherence property that makes a self-applicable rewrite-based language definition (in the sense of [6]) admit efficient bootstrapping.

A pleasant corollary: in a process pool with $n$ parallel components and $m$ enabled COMM redexes, the meta-evaluator's redex-discovery phase costs $O(n+m)$ communications, not the $O(n^2)$ that a naive self-interpreter would incur from trying every $(in, out)$ pair. The optimality of the underlying set automaton is exactly what bridges the gap.

# 7 Non-linear extensions

When some $L_i$ is non-linear — some variable repeats — the set automaton is not powerful enough to enforce the equality constraint: no regular tree automaton can recognise the language $\{t \mid t|_p \equiv t|_q\}$ for two fixed positions $p \neq q$ [5].

**Non-linear partition and consistency.** For a pattern $\ell$, the *non-linear partition* $\pi(\ell) \subseteq \mathcal{P}(\mathcal{E}(\ell))$ is the partition of the variable positions of $\ell$ by variable identity: $p, q \in \mathcal{E}(\ell)$ lie in the same class of $\pi(\ell)$ iff $\ell|_p$ and $\ell|_q$ are the same variable. A pattern is linear iff every class of $\pi(\ell)$ is a singleton; non-linearity is the presence of a class of size $\geq 2$. A term $t$ is *consistent* with $\pi(\ell)$ at position $p \in \mathcal{D}(t)$ iff for every class $P \in \pi(\ell)$ and every $q_1, q_2 \in P$, $t|_{p \cdot q_1} \equiv t|_{p \cdot q_2}$. A linear pattern is vacuously consistent.

**Pre-matches and the ambiguous/enabled/disabled trichotomy.** A *pre-match* is a pair $(\ell, p)$ such that $t|_p$ matches $\ell$ up to variable identification (i.e. $t|_p$ matches the linearisation of $\ell$). Bouwman–Erkens delegate consistency checking to the rewrite procedure, classifying every pre-match as *ambiguous* (consistency unchecked), *enabled* (consistency verified), or *disabled* (consistency violated); only enabled pre-matches may fire, and inner firings move pre-matches whose dependent subterms have changed back to *ambiguous*, where they will be re-checked [1, Algorithm 2]. We mirror this in the channel scheme by introducing auxiliary *consistency channels*.

**Definition 7.1** (Consistency receive)**.** Let $L$ be a pattern with non-linear partition $\pi(L)$. For each non-singleton class $P \in \pi(L)$ with $P = \{q_1, \ldots, q_k\}$ $(k \geq 2)$, define a guarded receive

$$matchP(t) \;=\; \mathsf{for}\,(z_1, \ldots, z_k \Leftarrow @t|_{q_1}, \ldots, @t|_{q_k})\,\{\, \mathit{if} \bigwedge_{i=2}^{k} z_1 =_{\mathcal{N}} z_i \;\mathit{then\ enable\ else\ disable}\,\}$$

where $=_{\mathcal{N}}$ is the rho-calculus name equality of §2.

The compiled non-linear rule then becomes

$$\mathsf{let}\ tc = \ulcorner T_M(K) \urcorner\ \mathsf{in}\ \mathsf{for}\,((\llbracket L_1 \rrbracket, \ldots, \llbracket L_n \rrbracket) \Leftarrow tc) \\ \{\, (\Big|\Big|_{i, P \in \pi(L_i), |P| \geq 2} matchP(L_i)) \mid \mathit{enable\text{-}guarded}(tc!(\llbracket K' \rrbracket(\ldots))) \,\},$$

where *enable-guarded*$(P)$ denotes the process that proceeds as $P$ once one *enable* token has been received per non-singleton class on a dedicated coordination channel, and remains blocked otherwise. The send to fire the rule is emitted only if all consistency receives report *enable*. This is exactly the *ambiguous* $\to$ *enabled* transition of [1, Algorithm 2, lines 12–17], lifted to channel form. The *disabled* state holds via the absence of the enable signal.

*Remark* 7.2 (Re-checking on inner firing). [1, Algorithm 2, lines 24–31] re-classifies a disabled or enabled match as ambiguous if an inner rewrite changes a subterm referenced by a non-singleton class of $\pi(L)$. In the channel scheme, the analogous behaviour is automatic: on inner firing, the inner rule's send pushes a fresh value to $tc(K_{\text{in}})$, which causes the enclosing *matchP* receives to re-fire with the new value. No explicit administrative loop is required, modulo the persistent-receive semantics of $\Leftarrow$.

# 8 Caveats and future work

**Look-ahead.** The set automaton requires a look-ahead bounded by the maximum LHS depth [1, §1.1]; equivalently, the channel $tc(K)$ may suspend before observing the entire surface of $K$, waiting for hole content to determine which sub-pattern to match. This corresponds, in implementation terms, to "intermediate channels" that exist between $tc(K)$ and $tc(K_{\text{in}})$. Their construction is mechanical (unfold step 2 of Construction 4.5 until either a match or a hole is reached) but they should be made explicit in any concrete implementation, as they bear on memory cost.

**Canonical state choice.** The state-labelling function $L : \mathcal{S} \to \mathbb{P}$ is, in [1], chosen non-deterministically among the positions of root match obligations. For $tc(\cdot)$ to be a function rather than a relation, the choice must be fixed at automaton-construction time; a lexicographic policy on positions is the obvious canonical choice and is what one would implement.

**Dynamic rule sets.** A fully-reflective GSLT — e.g. MeTTaIL [6] — permits new rewrite rules to be introduced at runtime. The construction of §4 assumes a closed set $\mathcal{L}$ to build the automaton from. Two extension strategies are available: incremental automaton update (recompute $\delta$ for the affected states as new patterns are added) or stratified compilation (a fresh sub-automaton per dynamic rule batch, dispatched by an outer name-equality check). The former preserves the optimality theorems but at higher construction cost; the latter is cheaper but degrades (O3) to a coarser partition that may distinguish a context $K$ across batches.

**Concurrent firing.** The rho calculus permits concurrent application of non-conflicting rules in a way that the imperative model of [1] does not. The $R_{\mathsf{dep}}$-partition at the heart of the set-automaton construction is exactly an analysis of independent sub-matchings; under the channel encoding it becomes an analysis of independent sub-channels that may be serviced in parallel. A precise statement and proof of a parallel-firing theorem — "two contextual rewrites whose outer contexts share no $R_{\mathsf{dep}}$-class fire concurrently" — is a natural follow-up to this paper.

**Right-hand-side precompilation.** [1, §8] suggest precompiling the configuration tree fragments arising from right-hand sides, so that the symbols of $R$ need not be matched again after substitution. The channel-side analogue is to precompile the trace $T_M(R)$ (with holes at the variables of $R$) and have the inner rule's send emit not $[\![ R ]\!]$ but a process already partially matched, attached at the trace's bud channels. The optimisation is an adaptation of Construction 4.5 to right-hand sides and is straightforward.

# 9 Conclusion

We have given a closed-form construction of the channel $tc(K)$ in the contextual-rewrite translation scheme of Definition 3.1, identified it with the canonical reflection of a set-automaton trace, and proved that it is optimal in three precise senses inherited from the Bouwman–Erkens framework. The construction is parametric in the state encoding, in the choice of dependency relation (and hence of strategy), and in the rho-calculus reflection bracket; this parametricity is what makes the polymorphic $\mathsf{let}\ tc = [\![ K ]\!]\ \mathsf{in}\ \cdots$ step — the original locus of the question — precise, computable, and theory-grounded. The non-linear extension and the dynamic-rule extension are sketched and left as engineering follow-ups.

**Acknowledgements.** The author thanks Mike Stay, Brian Beckman, and Hannah Adams for ongoing discussions that shaped the F1R3FLY platform context in which this work sits, and acknowledges the Bouwman–Erkens paper [1] as the immediate technical foundation.

# References

[1] M. Bouwman and R. Erkens. Term rewriting based on set automaton matching. arXiv:2202.08687, 2022.

[2] R. Erkens and J.F. Groote. A set automaton to locate all pattern matches in a term. In *Theoretical Aspects of Computing – ICTAC 2021*, LNCS 12819, Springer, pp. 67–85, 2021.

[3] F. Baader and T. Nipkow. *Term Rewriting and All That.* Cambridge University Press, 1998.

[4] L.G. Meredith and M. Radestock. A reflective higher-order calculus. *Electronic Notes in Theoretical Computer Science*, 141(5):49–67, 2005.

[5] H. Comon, M. Dauchet, R. Gilleron, C. Löding, F. Jacquemard, D. Lugiez, S. Tison, and M. Tommasi. Tree automata techniques and applications. `http://tata.gforge.inria.fr/`, 2007.

[6] L.G. Meredith. The MeTTaIL language: defining languages as triples of syntactic types, equations, and rewrites. F1R3FLY.io technical report and prototype, `https://github.com/F1R3FLY-io/mettail-rust`, 2025.